

CSCE 46103/56103 Artificial Intelligence – Fall 2026

Adversarial Search: Minimax, Alpha–Beta Pruning, and MCTS

Jungtaek Kim

`jungtaek.kim@uark.edu`

University of Arkansas

September 10, 2026

Today's Topics

- ▶ **Announcement of Assignment 2**
- ▶ **Minimax:** Choose the best move against an optimal opponent.
- ▶ **Alpha–beta pruning:** Skip branches that cannot change the decision.
- ▶ **Monte Carlo tree search (MCTS):** Estimate promising moves through sampled simulations.

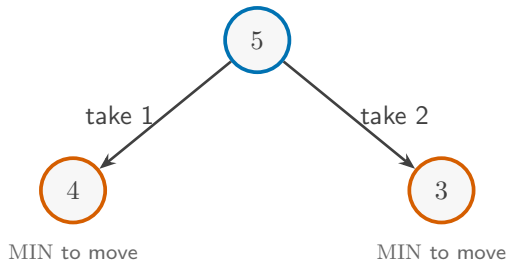
Opening Challenge: Best Move Against Best Defense



Five stones remain.

Rules

1. **MAX** moves first.
2. Each player removes one or two stones.
3. The player taking the final stone wins.



Prediction

Should **MAX** take **one** stone or **two** stones if **MIN** responds optimally?

Learning Objectives

By the end of this deck, you should be able to:

1. formulate a deterministic two-player game as a search problem;
2. compute minimax values and select an optimal action;
3. trace alpha-beta bounds and justify a cutoff;
4. trace one MCTS iteration and interpret UCT selection; and
5. distinguish exact backup, safe pruning, and statistical sampling.

Values

What can optimal
play force?

Bounds

What cannot change
the result?

Samples

Where should
computation go?



Single-Agent and Adversarial Search

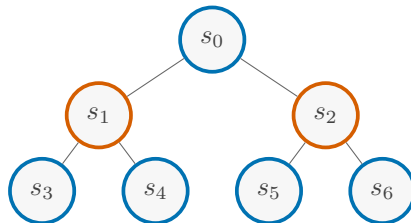
Single-Agent Search

The agent selects every action along the proposed path.



Adversarial Search

The opponent selects the reply at alternating levels.



A good move must survive the opponent's best reply.

Game-Search Model

Component	Meaning
s_0	initial state
ToMove(s)	player choosing next
Actions(s)	legal actions
Result(s, a)	successor after action a
Terminal(s)	whether play has ended
Utility(s, MAX)	terminal value for MAX

Five-Stone State

$$s = (n, p)$$

n : stones remaining

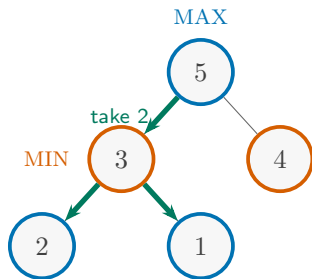
$p \in \{\text{MAX}, \text{MIN}\}$: player to move

Utility Convention

$$U = \begin{cases} +1, & \text{MAX wins,} \\ -1, & \text{MIN wins.} \end{cases}$$

Deterministic • two-player • alternating-turn • perfect-information • zero-sum

Strategies, Moves, and Ply



Move

One action selected at the current state.

Ply

One action by one player; one level deeper in the tree.

Strategy

A contingent plan specifying what to do after every possible opponent reply.

Choosing “take 2” is a move; planning for both replies from state 3 is part of a strategy.

Minimax Values

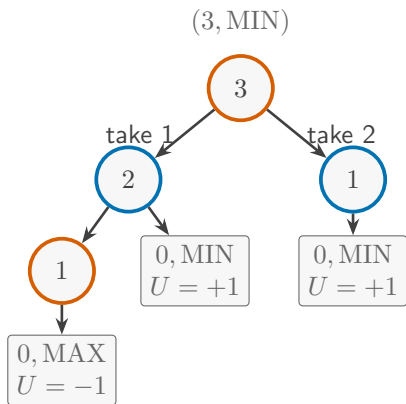
One Fixed Perspective

V takes one complete state: in the stone game, $s = (n, p)$ and $V(s) = V((n, p))$. The player p identifies whose turn it is. Every value is utility for **MAX**; **MIN** chooses the smallest successor value.

$$V(s) = \begin{cases} \text{Utility}(s, \text{MAX}), & \text{Terminal}(s), \\ \max_{a \in \text{Actions}(s)} V(\text{Result}(s, a)), & \text{ToMove}(s) = \text{MAX}, \\ \min_{a \in \text{Actions}(s)} V(\text{Result}(s, a)), & \text{ToMove}(s) = \text{MIN}. \end{cases}$$

Search moves **downward**; values move **upward**.

From Terminal Utilities to Minimax Values



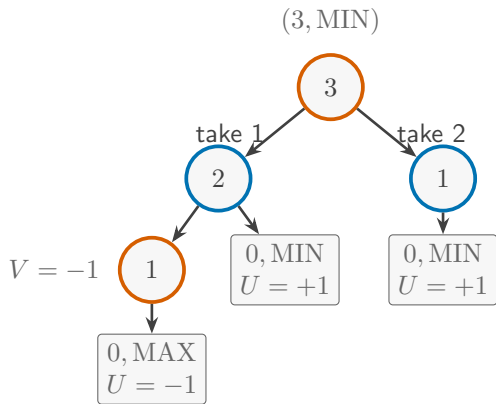
1. Read Terminal Utilities

At $n = 0$, the recorded next player has already lost: the other player took the final stone.

$$V((0, \text{MAX})) = -1,$$

$$V((0, \text{MIN})) = +1.$$

From Terminal Utilities to Minimax Values

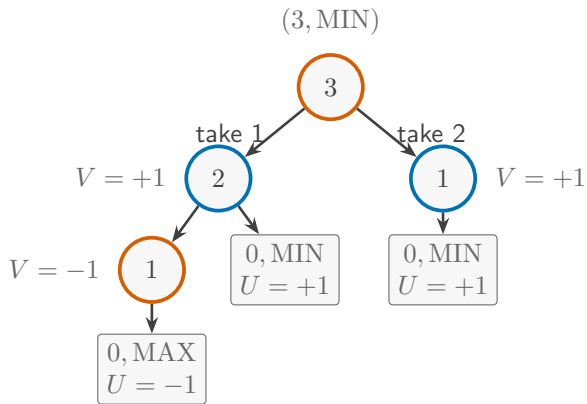


2. Resolve the Deepest MIN Node

From (1, MIN), the only action takes the final stone.

$$V((1, \text{MIN})) = \min\{-1\} = -1.$$

From Terminal Utilities to Minimax Values



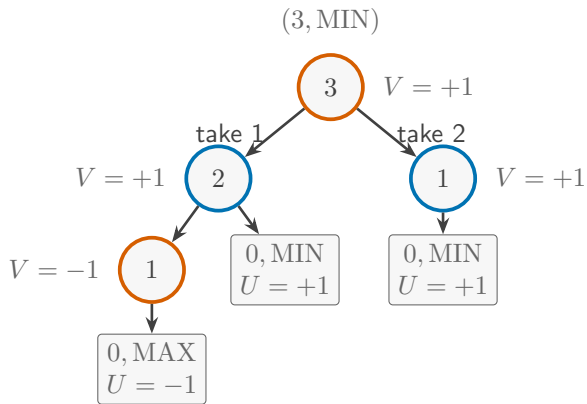
3. Back Up the MAX Nodes

MAX chooses the larger successor value.

$$V((2, \text{MAX})) = \max\{-1, +1\} = +1,$$

$$V((1, \text{MAX})) = \max\{+1\} = +1.$$

From Terminal Utilities to Minimax Values



4. Back Up the MIN Root

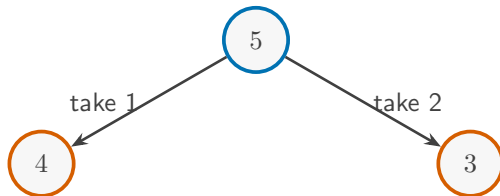
MIN chooses the smaller successor value.

$$V((3, \text{MIN})) = \min\{+1, +1\} = +1.$$

Either reply leaves MAX a forced win.

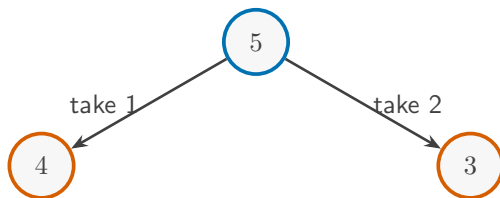
Bottom-Up Value Summary

Stones n	0	1	2	3	4	5
$V((n, \text{MAX}))$	-1					
$V((n, \text{MIN}))$	+1					



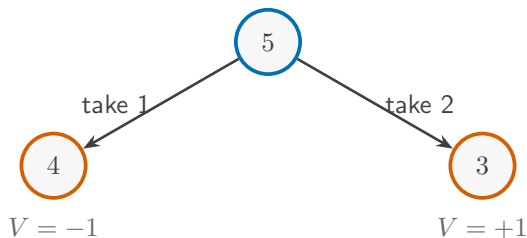
Bottom-Up Value Summary

Stones n	0	1	2	3	4	5
$V((n, \text{MAX}))$	-1	+1	+1			
$V((n, \text{MIN}))$	+1	-1	-1			



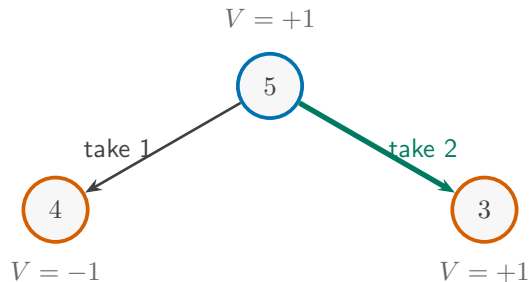
Bottom-Up Value Summary

Stones n	0	1	2	3	4	5
$V((n, \text{MAX}))$	-1	+1	+1	-1	+1	
$V((n, \text{MIN}))$	+1	-1	-1	+1	-1	



Bottom-Up Value Summary

Stones n	0	1	2	3	4	5
$V((n, \text{MAX}))$	-1	+1	+1	-1	+1	+1
$V((n, \text{MIN}))$	+1	-1	-1	+1	-1	-1



MAX removes **two** stones: $(3, \text{MIN})$ is losing for **MIN**.

Minimax Search

Recursive Value

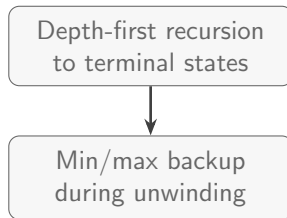
```
1: function Value( $s$ )
2: if Terminal( $s$ ) then
3:   return Utility( $s$ , MAX)
4: else if ToMove( $s$ ) = MAX then
5:   return  $\max_{a \in \text{Actions}(s)} \text{Value}(\text{Result}(s, a))$ 
6: else
7:   return  $\min_{a \in \text{Actions}(s)} \text{Value}(\text{Result}(s, a))$ 
8: end if
```

Root Decision

$$a^* = \arg \max_a V(s_a),$$

$$s_a = \text{Result}(s_0, a).$$

where $a \in \text{Actions}(s_0)$



Minimax Complexity and Guarantees

Cost

For branching factor b and maximum depth m ,

$$\text{time} = O(b^m).$$

Space is $O(m)$ with one-at-a-time successor generation, or $O(bm)$ when actions are generated together.

Guarantee

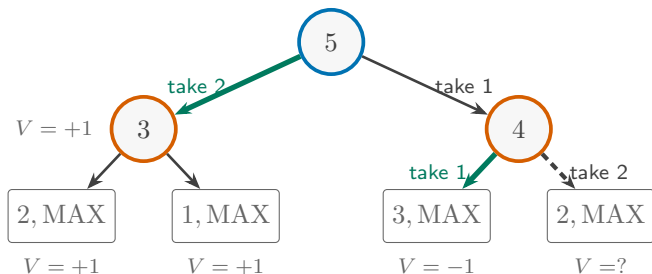
For a finite tree searched to terminal states, minimax returns an optimal action against an optimal opponent.

Important Assumption

Minimax protects against the best defense; it does not model which mistakes a particular opponent is likely to make.

Exact backup is conceptually clean and computationally expensive.

Irrelevant Subtrees



1. Exploring **take 2** proves that MAX can obtain $+1$.
2. Under **take 1**, MIN finds a reply worth -1 to MAX.

Question

Must we evaluate MIN's other reply under state 4?

A branch matters only if it can change an ancestor's decision.

Alpha and Beta Bounds

Alpha: MAX's Floor

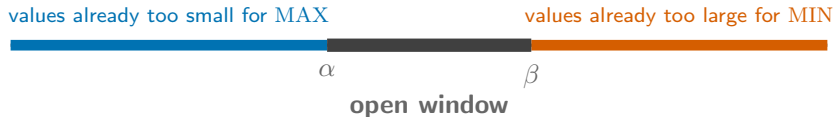
α = best value already secured by MAX

Along the current path, MAX can obtain **at least** α .

Beta: MIN's Ceiling

β = best limit already secured by MIN

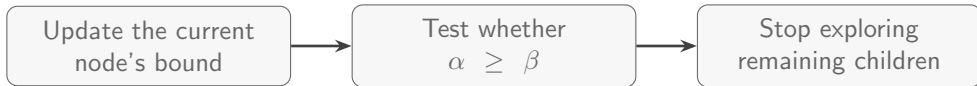
Along the current path, MIN can hold MAX to **at most** β .



Initialize $\alpha = -\infty$ and $\beta = +\infty$ at the root.

Cutoff Conditions

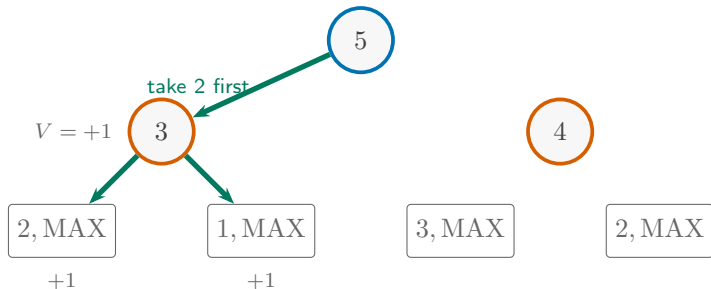
Node	Value update	Cutoff	Reason
MAX	$v \leftarrow \max(v, \text{child});$ $\alpha \leftarrow \max(\alpha, v)$	$v \geq \beta$	MIN already has a no-larger alternative
MIN	$v \leftarrow \min(v, \text{child});$ $\beta \leftarrow \min(\beta, v)$	$v \leq \alpha$	MAX already has a no-smaller alternative



Path Dependence

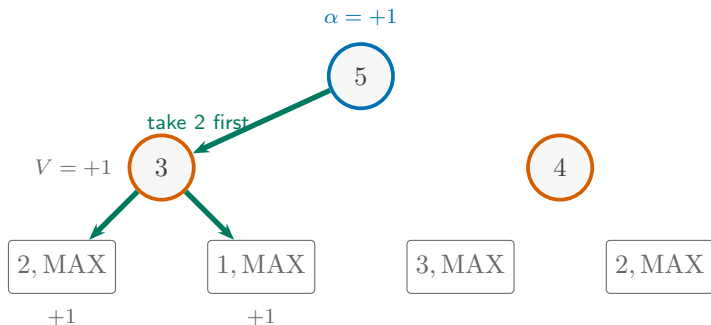
The bounds summarize alternatives on the current ancestor path; they are not permanent labels attached to a state. Each recursive call receives local copies of α and β . A child returns v ; its local bound updates do not directly change the parent's bounds.

Alpha-Beta Pruning Trace



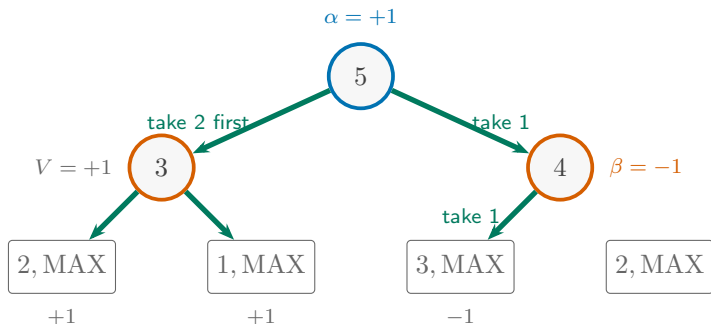
The take-2 branch returns +1 after both replies are resolved.

Alpha-Beta Pruning Trace



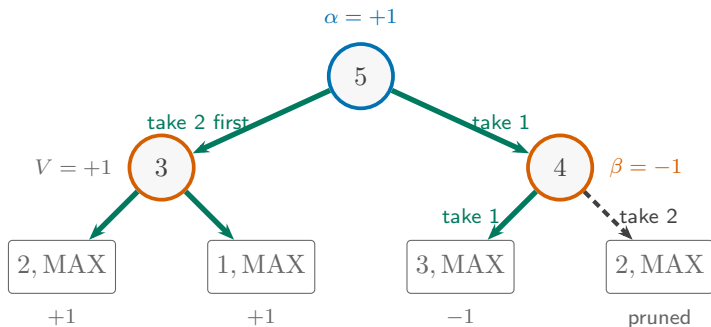
MAX records a guaranteed value of +1 at the root.

Alpha-Beta Pruning Trace



In the take-1 branch, MIN finds a reply with value -1 .

Alpha-Beta Pruning Trace



$\beta = -1 \leq \alpha = +1$
The window
is closed.

MIN will never choose a larger value, so its remaining reply cannot help MAX.

Alpha-Beta Search

MAX-VALUE(s, α, β)

```
1: if Terminal( $s$ ) then
2:   return Utility( $s$ , MAX)
3: end if
4:  $v \leftarrow -\infty$ 
5: for  $a \in \text{Actions}(s)$  do
6:    $v \leftarrow \max\{v, \text{MinValue}(\text{Result}(s, a), \alpha, \beta)\}$ 
7:   if  $v \geq \beta$  then
8:     return  $v$ 
9:   end if
10:   $\alpha \leftarrow \max\{\alpha, v\}$ 
11: end for
12: return  $v$ 
```

MIN-VALUE(s, α, β)

```
1: if Terminal( $s$ ) then
2:   return Utility( $s$ , MAX)
3: end if
4:  $v \leftarrow +\infty$ 
5: for  $a \in \text{Actions}(s)$  do
6:    $v \leftarrow \min\{v, \text{MaxValue}(\text{Result}(s, a), \alpha, \beta)\}$ 
7:   if  $v \leq \alpha$  then
8:     return  $v$ 
9:   end if
10:   $\beta \leftarrow \min\{\beta, v\}$ 
11: end for
12: return  $v$ 
```

Start with $\text{MaxValue}(s_0, -\infty, +\infty)$ and retain the root action that improves v .

Cutoff Correctness

Five-Stone Cutoff

After the take-2 action has established $\alpha = +1$, let x be the unknown value of the unexamined reply under take 1:

$$\max(+1, \min(-1, x)) = \max(+1, -1) = +1, \quad x \in \{-1, +1\}.$$

Preserved

- ▶ the root minimax value;
- ▶ at least one optimal root action.

Not Established

A pruned subtree is not necessarily bad in isolation. It is irrelevant given an alternative already available to an ancestor.

A cutoff may return a sufficient bound without resolving the node's exact standalone value.

Move Ordering and Search Effort

Take 2 First

1. Establish root $\alpha = +1$.
2. Enter the take-1 branch.
3. Cut off as soon as MIN finds -1 .

Earlier cutoff

Take 1 First

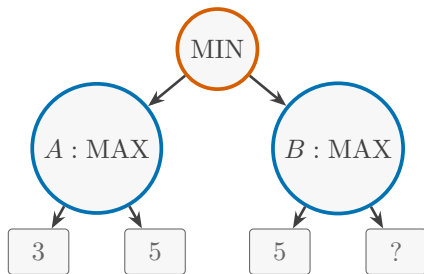
1. Establish only $\alpha = -1$.
2. Enter the take-2 branch.
3. Resolve more descendants before proving $+1$.

Less pruning

	Worst ordering	Perfect ordering
Time	$O(b^m)$	$O(b^{m/2})$
Root result	same minimax value and an optimal action	

Guided Pruning Challenge

Search this separate game tree left to right, depth first, with $(\alpha, \beta) = (-\infty, +\infty)$.



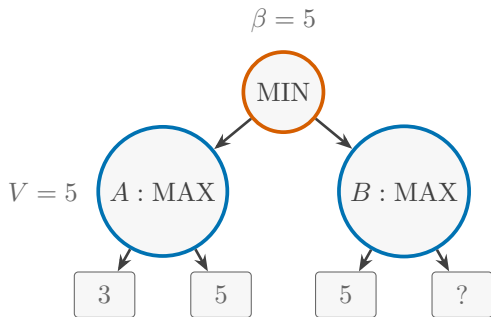
Circles are decision nodes; boxes contain terminal utilities for MAX.

Three Predictions

1. After A returns, what bound is passed to B ?
2. After B sees its first leaf, can it skip the other leaf?
3. Would returning 5 prove that $V(B) = 5$?

Guided Pruning Challenge

Search this separate game tree left to right, depth first, with $(\alpha, \beta) = (-\infty, +\infty)$.



1. Pass an Upper Bound

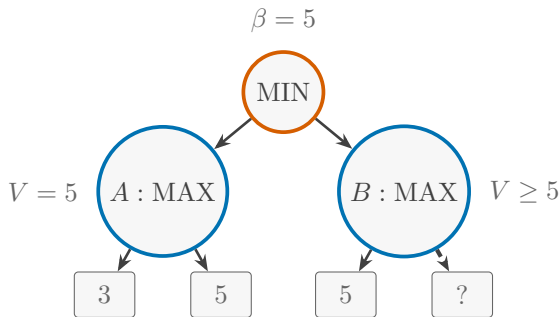
A returns $\max(3, 5) = 5$. The MIN root can therefore hold MAX to at most 5.

B receives $(\alpha, \beta) = (-\infty, 5)$.

Circles are decision nodes; boxes contain terminal utilities for MAX.

Guided Pruning Challenge

Search this separate game tree left to right, depth first, with $(\alpha, \beta) = (-\infty, +\infty)$.



2. Cut Off at Equality

B is a MAX node. Its first leaf establishes $v = 5$, so

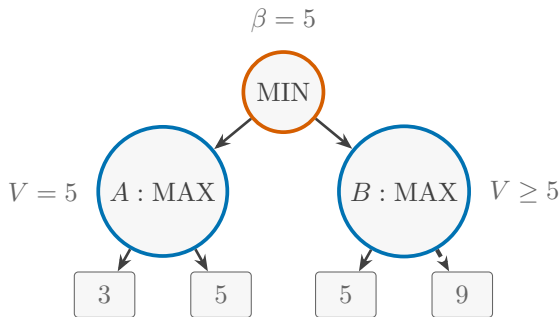
$$v \geq \beta \iff 5 \geq 5.$$

Skip its remaining leaf and return 5 as a lower bound. The root already has A at 5.

Circles are decision nodes; boxes contain terminal utilities for MAX.

Guided Pruning Challenge

Search this separate game tree left to right, depth first, with $(\alpha, \beta) = (-\infty, +\infty)$.



3. Check the Skipped Leaf

Suppose the unexamined utility was 9.

$$V(B) = \max(5, 9) = 9.$$

The exact root value is still $\min(5, 9) = 5$, attained through A . The returned bound was sufficient.

Circles are decision nodes; boxes contain terminal utilities for MAX.

Depth Limits and Evaluation

Leaf Values with a Depth Limit

Search at most d plies ahead. At each search leaf, use

$$\text{leaf value}(s) = \begin{cases} \text{Utility}(s, \text{MAX}), & \text{Terminal}(s), \\ E(s), & \text{nonterminal state at the depth limit.} \end{cases}$$

$E(s)$ is a heuristic estimate of utility for MAX. Back up these values using the same min/max rules.

Depth Limits and Evaluation

Leaf Values with a Depth Limit

Search at most d plies ahead. At each search leaf, use

$$\text{leaf value}(s) = \begin{cases} \text{Utility}(s, \text{MAX}), & \text{Terminal}(s), \\ E(s), & \text{nonterminal state at the depth limit.} \end{cases}$$

$E(s)$ is a heuristic estimate of utility for MAX. Back up these values using the same min/max rules.

What Alpha–Beta Preserves

For the same depth limit, evaluation function, and search tree, alpha–beta preserves the truncated minimax value and an optimal action for that truncated tree.

Approximation and Horizon

Heuristic leaf values need not equal full-game minimax values. A decisive event just beyond the cutoff can be missed: the horizon effect.

Sampling Motivation

Five-Stone Game

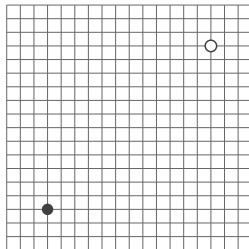


20 tree nodes
8 terminal histories

Complete backup is practical.

Heuristic Evaluation
Estimate from state features

Go



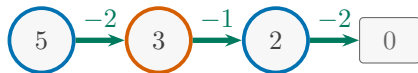
- ▶ up to 361 opening placements;
- ▶ enormous game tree;
- ▶ need to estimate nonterminal values.

or

Rollout Evaluation
Estimate from
sampled returns

Rollouts and Playout Policies

One Rollout



MAX wins this simulation, so the sampled return is $z = +1$.

Value Estimate

For a node visited $N > 0$ times,

$$U = \sum_{i=1}^N z_i, \quad Q = \frac{U}{N}.$$

Every z_i is utility for MAX.

Uniform-Random Rollouts

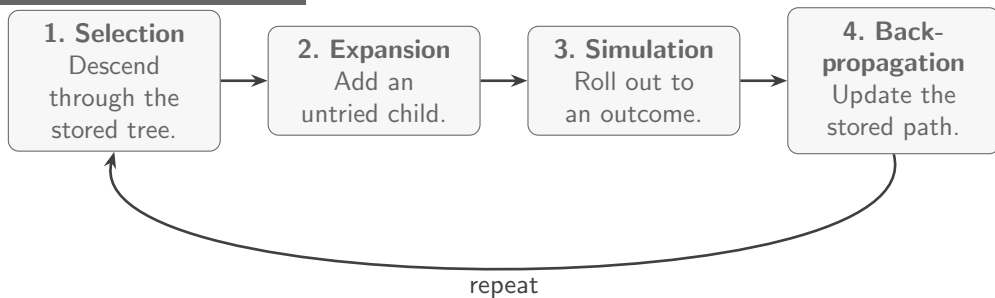
After fixing the root action, both players choose uniformly among legal moves.

Action	MAX win prob.	$\mathbb{E}[z]$
take 1	3/8	-0.25
take 2	3/4	+0.50

Policy Matters

These means describe random play. A stronger playing policy does not necessarily produce more accurate rollout estimates.

The MCTS Cycle



Stored Search Tree

Selection traverses recorded nodes; expansion adds one previously unrecorded child.

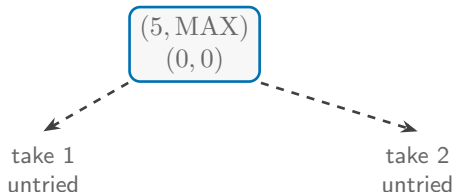
Unstored Rollout

Simulation follows a playout policy from the new child to a terminal outcome.

The new child and its ancestors receive the rollout result.
Transient rollout states are not added to the stored tree.

MCTS from the Root

State (n, p) ; statistics (N, U) ; all returns are for MAX.



1. Selection

Start with only the root:

$$(N, U) = (0, 0).$$

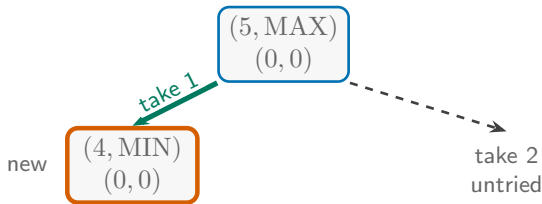
Selection stops here because the root has untried actions.

Before Any Simulation

Dashed arrows mark possible actions. No children are stored yet.

MCTS from the Root

State (n, p) ; statistics (N, U) ; all returns are for MAX.



2. Expansion

Choose take 1 first and add $(4, \text{MIN})$ with

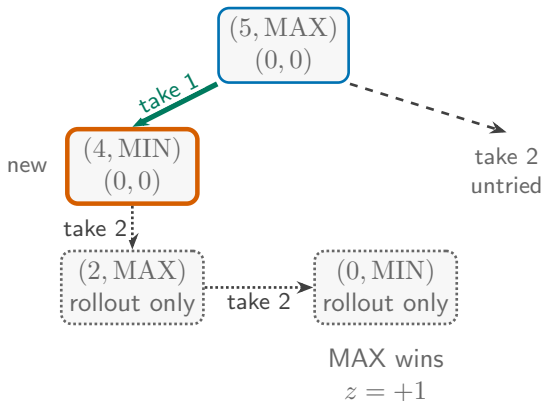
$$(N, U) = (0, 0).$$

First Visit Pending

The new child and root still have zero visits. Update them after the rollout.

MCTS from the Root

State (n, p) ; statistics (N, U) ; all returns are for MAX.



3. Simulation

MIN takes 2, then MAX takes the last 2 and wins:

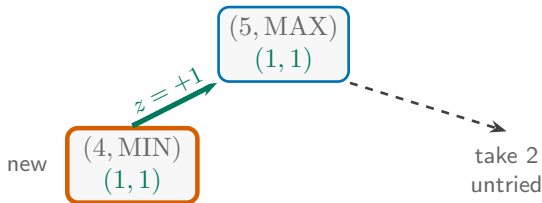
$$z = +1.$$

Rollout Only

The dotted states are temporary. Only the root and $(4, \text{MIN})$ are stored.

MCTS from the Root

State (n, p) ; statistics (N, U) ; all returns are for MAX.



4. Backpropagation

Update the new child and root with the same MAX return:

$$N \leftarrow N + 1,$$

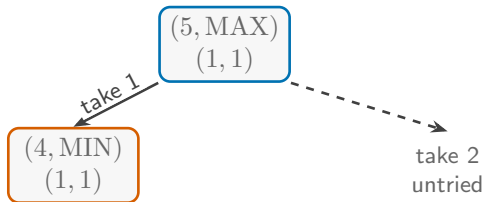
$$U \leftarrow U + 1.$$

After One Iteration

Both stored nodes have $(N, U) = (1, 1)$. Take 2 at the root remains untried.

MCTS: The Second Iteration

State (n, p) ; statistics (N, U) ; all returns are for MAX.



1. Selection

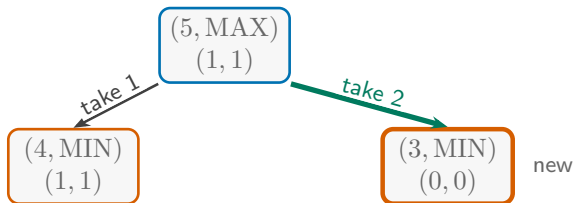
The root still has untried take 2, so selection stops at the root again.

Try Each Root Action

The first child has one visit. Try the other action before comparing the children.

MCTS: The Second Iteration

State (n, p) ; statistics (N, U) ; all returns are for MAX.



2. Expansion

Expand take 2 and add $(3, \text{MIN})$ with

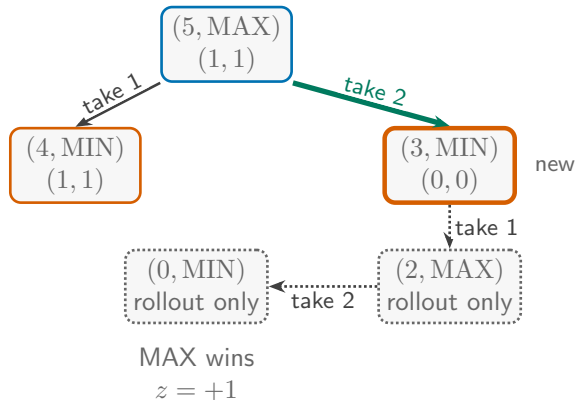
$$(N, U) = (0, 0).$$

First Visit Pending

Existing statistics stay unchanged until the rollout finishes.

MCTS: The Second Iteration

State (n, p) ; statistics (N, U) ; all returns are for MAX.



3. Simulation

MIN takes 1, then MAX takes the last 2 and wins:

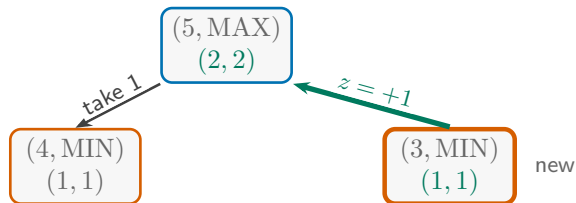
$$z = +1.$$

Rollout Only

The dotted states are temporary.
The stored tree has three nodes.

MCTS: The Second Iteration

State (n, p) ; statistics (N, U) ; all returns are for MAX.



4. Backpropagation

The root becomes $(2, 2)$ and the new child becomes $(1, 1)$. The other child stays $(1, 1)$.

Next Selection

Both root actions have been tried. Use UCT to choose a child next.

UCT Selection

Opponent-Aware Score

Let $t = \text{ToMove}(\text{parent}(n))$ be the player choosing child n . With $U(n)$ stored from MAX's perspective, maximize

$$\text{UCT}(n) = \underbrace{\sigma(t) \frac{U(n)}{N(n)}}_{\text{exploitation}} + \underbrace{C \sqrt{\frac{\ln N(\text{parent}(n))}{N(n)}}}_{\text{exploration}}, \quad \sigma(t) = \begin{cases} +1, & t = \text{MAX}, \\ -1, & t = \text{MIN}. \end{cases}$$

The exploration constant $C > 0$ controls the weight of the bonus.

Exploitation

Prefer outcomes favorable to the player choosing now.

Exploration

Give a larger bonus to actions with fewer visits.

First Visits

Try each untried action before applying UCT to its child; the formula requires $N(n) > 0$.

Worked UCT Choice

After 11 Completed Iterations

After nine more iterations with $C = \sqrt{2}$, this example reaches $(N, U)_{\text{root}} = (11, 7)$ at (5, MAX).

Action	Child state	(N, U)	U/N	Exploration bonus	UCT score
take 1	(4, MIN)	(3, 1)	0.333	$\sqrt{2 \ln(11)/3} = 1.264$	1.598
take 2	(3, MIN)	(8, 6)	0.750	$\sqrt{2 \ln(11)/8} = 0.774$	1.524

UCT selects take 1: its exploration bonus outweighs its lower empirical mean.

Worked UCT Choice

After 11 Completed Iterations

After nine more iterations with $C = \sqrt{2}$, this example reaches $(N, U)_{\text{root}} = (11, 7)$ at (5, MAX).

Action	Child state	(N, U)	U/N	Exploration bonus	UCT score
take 1	(4, MIN)	(3, 1)	0.333	$\sqrt{2 \ln(11)/3} = 1.264$	1.598
take 2	(3, MIN)	(8, 6)	0.750	$\sqrt{2 \ln(11)/8} = 0.774$	1.524

UCT selects take 1: its exploration bonus outweighs its lower empirical mean.

Opponent Selection at (4, MIN)

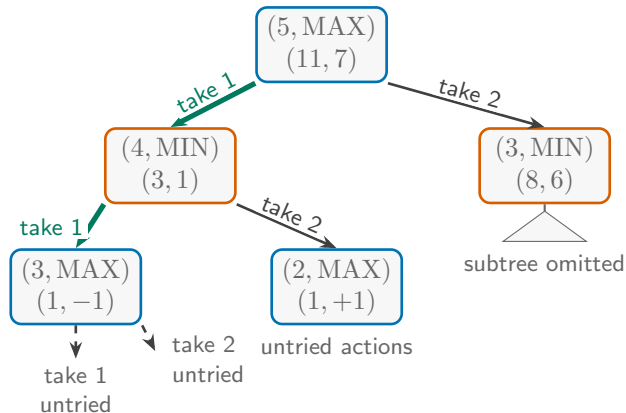
Take 1 reaches (3, MAX) with $(N, U) = (1, -1)$; take 2 reaches (2, MAX) with $(1, +1)$. Their bonuses match, and the exploitation terms are

$$-(-1) = +1 \quad \text{for take 1,} \quad -(+1) = -1 \quad \text{for take 2.}$$

UCT selects MIN's take-1 action, reaching (3, MAX).

MCTS: The Twelfth Iteration

State (n, p) ; statistics (N, U) ; all returns are for MAX.



1. Selection

UCT selects take 1 at both levels, reaching $(3, \text{MAX})$.

Visits at $(4, \text{MIN})$

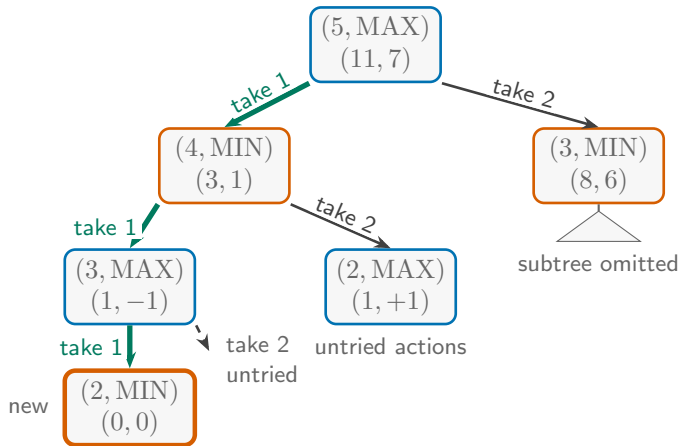
The initial rollout returned +1 before any children were stored.

Contribution	N	U
Creation rollout	1	+1
Child $(3, \text{MAX})$	1	-1
Child $(2, \text{MAX})$	1	+1
Parent total	3	+1

Child counts exclude that initial rollout.

MCTS: The Twelfth Iteration

State (n, p) ; statistics (N, U) ; all returns are for MAX.



2. Expansion

Add $(2, \text{MIN})$ as the only new stored node:

$$(N, U) = (0, 0).$$

Record its first visit after the rollout.

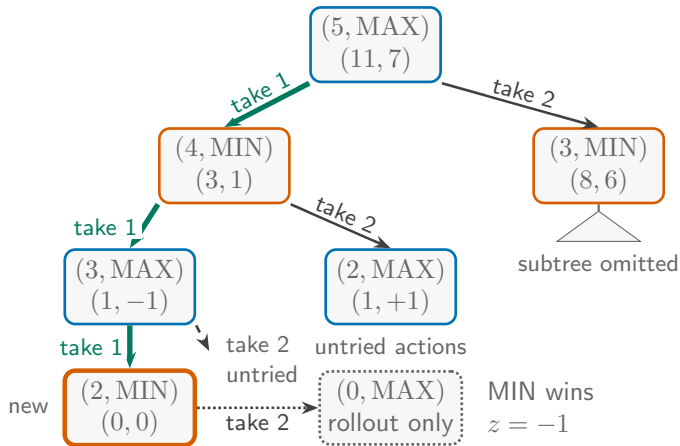
Remaining Action

Take 2 at $(3, \text{MAX})$ is still untried.

Solid edges connect stored nodes.

MCTS: The Twelfth Iteration

State (n, p) ; statistics (N, U) ; all returns are for MAX.



3. Simulation

From $(2, \text{MIN})$, MIN takes 2 and wins. MAX's sampled return is

$$z = -1.$$

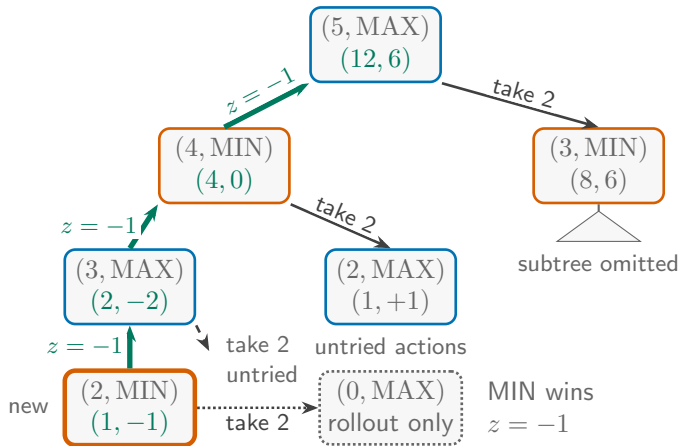
Rollout Only

The dotted edge continues the simulation. Its terminal state is not stored.

Statistics are updated after the rollout.

MCTS: The Twelfth Iteration

State (n, p) ; statistics (N, U) ; all returns are for MAX.



4. Backpropagation

Update the new child and every ancestor with the same MAX return:

$$N \leftarrow N + 1,$$
$$U \leftarrow U - 1.$$

Updated Stored Path

Green arrows carry $z = -1$ upward. Green statistics show the updated (N, U) .

Sibling statistics stay unchanged.

Stopping and Root Action

Stopping Rule

Stop when a preset time or iteration budget is exhausted. Here, suppose the budget is **12 completed iterations**: the update makes $N_{\text{root}} = 12$, so stop now.

After the $z = -1$ Update

The root now has $(N, U) = (12, 6)$.

Action	(N, U)	U/N	Next UCT
take 1	(4, 0)	0	1.115
take 2	(8, 6)	0.750	1.538

With more budget, the next UCT selection would favor take 2.

Full-tree minimax and alpha-beta prove take 2.
Finite-budget MCTS recommends it from sampled evidence.

When Computation Stops

Return the root action with the largest visit count:

$$8 > 4 \implies \text{take 2.}$$

Selection vs. Recommendation

UCT chooses where to sample next. Visit count chooses the final move in this algorithm.

Discussion: Choosing the Final Move

Question

Why does this algorithm choose the root action with the largest visit count N , rather than the largest mean return $Q = U/N$?

MCTS Algorithm

Monte-Carlo-Tree-Search(s_0)

```
1:  $T \leftarrow \text{Node}(s_0)$ 
2: while computation remains do
3:    $\ell \leftarrow \text{Select}(T)$ 
4:   if  $\text{Terminal}(\ell.\text{state})$  then
5:      $c \leftarrow \ell$ ;  $z \leftarrow \text{Utility}(\ell.\text{state}, \text{MAX})$ 
6:   else
7:      $c \leftarrow \text{Expand}(\ell)$ 
8:      $z \leftarrow \text{Simulate}(c)$ 
9:   end if
10:   $\text{Backpropagate}(c, z)$ 
11: end while
12: return root action with largest child visit count
```

Selection Contract

Stop at a terminal node or a node with an untried action. Otherwise descend using UCT.

Expansion Contract

Add one untried child with $N = U = 0$. A terminal child returns its utility immediately.

Finite Budget

Each completed iteration updates the tree; the recommended action can still be wrong.

Algorithm Comparison

	Minimax	Alpha-Beta	MCTS
Leaf evidence	terminal utilities or cutoff evaluations	same leaf values as minimax	sampled rollout returns
Tree work	full tree to the chosen horizon	pruned tree to the same horizon	adaptively grown partial tree
Result	exact for terminal search	same minimax result at the same horizon	finite-budget estimate
Main lever	branching factor and depth	move ordering	selection and playout policies
Main risk	exponential cost	little pruning under poor ordering	missed decisive replies

Search Horizon

With heuristic cutoff values, minimax and alpha-beta both approximate the full-game result.

Method-Selection Challenge

1. A small finite game tree fits in memory and every terminal utility is available.
2. A larger deterministic tree has a promising move-ordering heuristic, but the exact decision is still required.
3. A game has an enormous branching factor, cheap simulations, and no reliable static evaluation function.

Prompt

Choose the most appropriate mechanism for each case and justify the choice using the tree structure or evidence available.

Method-Selection Challenge

1. A small finite game tree fits in memory and every terminal utility is available.
2. A larger deterministic tree has a promising move-ordering heuristic, but the exact decision is still required.
3. A game has an enormous branching factor, cheap simulations, and no reliable static evaluation function.

Prompt

Choose the most appropriate mechanism for each case and justify the choice using the tree structure or evidence available.

1. Complete Backup

Minimax

2. Safe Pruning

Alpha-beta

3. Adaptive Sampling

MCTS

Opening Challenge Revisited

○○○○○ **MAX** should take **two** stones.

Minimax Proof

$$\max\{-1, +1\} = +1.$$

Complete backup proves the forced win.

Alpha-Beta Proof

$$\beta = -1 \leq \alpha = +1.$$

A remaining subtree cannot change the proof.

MCTS Evidence

$$N_{\text{take } 2} = 8 > 4 = N_{\text{take } 1}.$$

The sampled tree recommends the same move.

Adversarial-Search Takeaways

- ▶ **Values:** Minimax alternates max and min backups under optimal play.
- ▶ **Bounds:** Alpha–beta removes branches that cannot affect the minimax decision.
- ▶ **Samples:** MCTS grows a partial tree by balancing exploitation and exploration.

Thank You

Any questions?

References

- S. Gelly and D. Silver. Combining online and offline knowledge in UCT. In *Proceedings of the 24th International Conference on Machine Learning*, pages 273–280. ACM, 2007. doi: 10.1145/1273496.1273531. URL <https://davidstarsilver.wordpress.com/wp-content/uploads/2025/05/ijcai-07.pdf>.
- S. J. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4th edition, 2020. URL <https://aima.cs.berkeley.edu/>.